

One Log, Two Projections, One **Snapshot**

Eight patterns measured on one NATS server and one small Go binary. Each card states the mechanism, what it buys, what it costs, and the number the lab actually printed. Two of the cards exist because the lab got it wrong first.

THE QUESTION EVERY CARD SERVES

A truck's odometer lives only in a log. Rehydrating it to check one command — **how much does a snapshot buy you**, and what happens to the answer when you scale the consumer that folds the log?

THE RULE THAT ORDERS THE DECK

A projection is **disposable** and the log is not. Everything a projection does — caching, denormalising, snapshotting, folding on many workers — is a trade you make against the log, and **every one of those trades is paid for in order**.

THE EIGHT PATTERNS

- 01 · The log is the only source of truth
- 02 · Two projections, one log
- 03 · What a snapshot buys
- 04 · The ordered consumer that ate the measurement
- 05 · A fold is defined by order
- 06 · A worker pool buys throughput with correctness
- 07 · MaxAckPending is the loss dial
- 08 · Redelivery after AckWait is not recovery

01

SOURCING

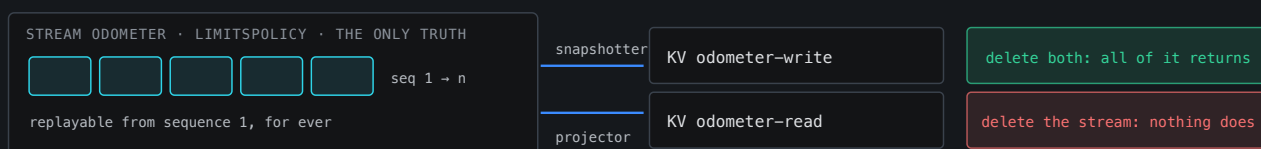
The log is the only source of truth

The stream holds the facts. Every KV bucket in this demo is a **fold of that stream** and holds nothing the stream does not. The test is brutal and cheap: delete every bucket and every consumer, restart, and see what comes back.

DECISION

Is a KV bucket ever allowed to be the truth? **No**. It is a cache of a fold. If losing it loses data, the design is wrong, not the bucket.

MECHANISM



- › **LimitsPolicy, never InterestPolicy.** InterestPolicy deletes a message once every consumer has acked it, and the replay then returns nothing — **with no error**. A stream you cannot replay is not an event source.
- › Measured 2026-09-16: both buckets and both durables deleted, both consumers refolded all **74 135** events from sequence 1. **Every KV document came back byte-identical**, `lastTripAt` included, to the nanosecond.
- › That only holds because `project()` uses the **event's own stream timestamp** and never the wall clock. A projection that reads the clock cannot be rebuilt — it can only be re-approximated.

WHAT IT BUYS

- › **A projection becomes disposable.** Change its shape, delete it, refold. No migration.
- › **Audit for free.** The sequence of transitions is the data, not a side table.
- › **Time travel.** Fold to sequence N and you have the state as of N.

WHAT IT COSTS

- › **The log only grows.** Retention is a product decision you now have to make.
- › **A rebuild is an outage for that read model** unless you fold into a second bucket and swap.
- › **Determinism is a constraint on your code for ever** — no clocks, no randomness, no lookups against data that also moves.

VERDICT

Event-source where history is itself a domain concern. Use plain CRUD where only the current value matters — the deciding question is **"does anything need to replay this"**, not "does it change".

02

CQRS

Two projections, one log

The write side and the read side fold the same stream into different shapes, in different processes, on different durables. They are not two copies of one thing. They disagree on purpose, and the disagreement is the whole of CQRS.

DECISION

Should the write-side snapshot just reuse the read model? **No.** The aggregate must carry only what a **command** is judged against. Everything else belongs to the reader.

MECHANISM

- › **odometer-write** holds `{state, lastSeq}` for the **aggregate**. It answers one question: may this command be appended?
- › **odometer-read** holds the **denormalised** view a screen wants — total kilometres, last trip, plate, status.
- › Measured 2026-09-16: after a full rebuild both sides landed on the same **lastSeq** for every vehicle, and the same **status** and **plate**.
- › **They do not agree on `totalKm`, and must not.** The write-side aggregate has no such field: **Travelled** leaves it unchanged, because no command is ever judged against a distance.



WHAT IT BUYS

- › **The write-side snapshot stays tiny for ever.** Ten thousand trips move the read model and leave the aggregate byte-for-byte identical. That is why Card 03's number does not decay.
- › **Read shapes are free to change.** A new screen is a new fold, not a schema change.
- › Write and read scale apart, and fail apart.

WHAT IT COSTS

- › **Two folds to keep correct**, and two durables to operate.
- › **The read model is eventually consistent.** A command that succeeds is not yet visible on the screen that triggered it.
- › "Which bucket is right?" becomes a question people ask. The answer is neither — Card 01.

VERDICT

Put a field on the aggregate only when a **rule** reads it. If the field exists to be displayed, it belongs to the read model, and keeping it out is what keeps rehydration cheap.

03

PERFORMANCE

What a snapshot buys

Rehydrating from sequence 1 is **linear in the length of the log**. Rehydrating from a snapshot is **one KV get plus the tail**, and does not care how long the log is. That is the whole finding, and the shape matters more than the multiple.

DECISION Is a snapshot worth the second moving part? **Yes, past a few thousand events** — and the reason is the slope, not today's ratio.

MEASURED · 2026-09-16 · NATS 2.14.3 IN DOCKER ON A LAPTOP · 10 001 EVENTS ON ONE VEHICLE · BEST OF THREE

REHYDRATION	EVENTS READ	TIME	GROWS WITH THE LOG?
From sequence 1, no snapshot	10 001	25 ms	Yes, linearly
From the snapshot, then the tail	0	1.1 ms	No

COST OF ONE REHYDRATION AS THE LOG GROWS

- › **About 23x here, and it grows.** Take the log to 100 000 events and the left figure goes up tenfold while the right one does not move.
- › **The snapshot is always stale.** The write consumer is asynchronous, so it trails the log. Rehydration reads the snapshot and then replays from `lastSeq + 1`. Measured mid-catch-up, the same command read **2 900 events of tail in 12 ms**.

WHAT IT BUYS

- › **A constant-time write path** on a log that is not constant.
- › It is cheap to add later — the snapshot is derived, so there is nothing to migrate.

WHAT IT COSTS

- › **Code that stops at the snapshot is a bug, not a shortcut.** You must always replay the tail.
- › A second consumer to run and watch.
- › Below a few thousand events it buys a millisecond and costs you a component.

04

GOTCHA · METHOD

The ordered consumer that ate the measurement

This demo's headline number was wrong for two days, and it was wrong in the direction that flatters the demo. The replay was not slow because replays are slow. It was slow because the client was creating **one consumer per event**.

DECISION

Which call reads a stream in bulk? `Messages()`, not `Next()`. On an ordered consumer, `Next()` resets the consumer on every call.

MECHANISM

- › An **ordered** consumer is ephemeral and self-healing: the client deletes the server-side consumer and creates a new one whenever it re-positions. `Next()` re-positions every time it is called.
- › It is visible on the server, because an ordered consumer's name carries a serial number that climbs once per message:

```
5yL5cUQZHsfSpLY8WvMDXR_32469 delivering stream sequence 32478
```

- › Before the fix the replay of 10 001 events took **8.2 s**. After it, **25 ms**. About **99% of the original figure was consumer bookkeeping** and about 1% was reading a log.
- › `Messages()` creates **one** consumer and pulls batches over it — which is what the client's own documentation on `Next()` tells you to do.
- › **The lesson survived the correction.** Replay is linear, a snapshot is constant. What changed is that the number now measures that, and not a mistake in the demo.

HOW TO CATCH IT YOURSELF

- › **Watch the server while you measure.** `nats consumer ls` during a replay — a name that changes per message is this bug.
- › **Distrust a flattering number.** A result that makes your design look good is the one to re-derive first.
- › Sanity-check the rate: 10 000 small events in 8 s is ~1 200/s, which is far below what one NATS server does. The arithmetic was the tell.

WHY IT IS EASY TO MISS

- › **No error, no warning.** The code is correct, just used against the wrong consumer kind.
- › It gets **worse with more data**, so it reads exactly like the linear cost you expected to find.
- › A unit test cannot see it. Only the server, or the clock, can.

VERDICT

Publish the correction next to the number. A lab whose numbers can only go up is not measuring — the retraction is the part that makes the rest of the deck worth reading.

05

CORRECTNESS

A fold is defined by order

A projection is a fold over a sequence. Once you allow a second reader, events can arrive out of sequence, and the fold has to say what it does about that. There are exactly three answers, and the demo makes each one a written rule.

DECISION

How does an event compare with the state already folded? **Ahead, equal, or behind** — and each one has a different, permanent answer.

THE COMPARISON, AND THE THREE ANSWERS · CQRS/DOMAIN.GO

THE EVENT IS...	RULE	WHAT THE FOLD DOES	WHY
Ahead of the folded state	BR-OD06	Apply it	This is the normal case.
Equal — seen before	BR-OD07	Ignore it, ack it	A redelivery is a duplicate, not new work.
Behind the folded state	BR-OD08	Refuse it — ErrOutOfOrder	Applying it would move the odometer backwards.

> BR-OD09 — an undecodable event is permanent. ErrUndecodable is not retried. Redelivering a message the code cannot parse only produces the same failure at ack-wait intervals forever.

> The rules live in domain.go and nowhere else. No rule enforcement sits in the CLI, the write side, the read side or the HTTP shim — so there is exactly one place to read them and exactly one place to test them.

> Idempotence is not enough on its own. BR-OD07 makes a repeat harmless. BR-OD08 is the one that stops a late event from corrupting a monotonic value.

WHAT THIS GIVES YOU

> Out-of-order delivery becomes a **named, tested condition**, not a mystery.

> A refusal is **visible**: it is counted, so a wrong fold shows up as a number.

> Redelivery costs nothing when the fold is already past the event.

WHAT IT DOES NOT GIVE YOU

> **A refused event is lost work**, not repaired work. The fold is correct, but incomplete.

> It only helps a value where order *means* something. A counter you can add to does not need it.

> You must carry the compared value (here, the last sequence) in the projection itself.

06

SCALING

A worker pool buys throughput with correctness

Adding workers to one durable consumer makes the fold faster. It also makes it wrong. Both halves are measured on the same run, on the same log, and neither is a surprise once you see where the order is lost.

DECISION

Can you scale a projection by adding workers? **Only if the fold is order-free.** An odometer is not, so the speed is paid for in dropped events.

MEASURED · 2026-09-15 · NATS 2.14.3 · ODOMETER_POOL , 10 029 EVENTS, THREE VEHICLES, -DRAIN

WORKERS	TIME	RATE	FOLDED	DROPPED (OUT OF ORDER)
1	23.5 s	426/s	10 029	0
2	12.2 s	824/s	10 025	4
4	6.3 s	1 600/s	6 947	3 082 (31%)
8	4.8 s	2 100/s	4 338	5 691 (57%)

> 3.7x faster at four workers, and a third of the log refused.

Those are the same run.

> One worker drops nothing.

One worker cannot race itself, so it is the control row and it proves the fold itself is correct.

> Where the order is lost:

two events for the SAME vehicle land on two workers, and the later one finishes first. The fold then sees an event behind its state and refuses it (BR-OD08).

> Two dials control how often that happens.

The key gap — how far apart two events of one vehicle sit in the log — and the in-flight window, MaxAckPending . The demo seeds three vehicles round-robin, so the gap is small on purpose; at ten vehicles the gap was wide enough that nothing ever raced and the lesson taught nothing.

WHEN A POOL IS RIGHT

> The fold is commutative — a sum, a set, a last-write-wins cache.

> Work is partitioned by key, one consumer per key range, so no two workers share a key.

> You care about drain time more than about per-key ordering.

WHEN IT IS WRONG

> Any monotonic value — an odometer, a balance, a version.

> Any state machine with transitions that must not be re-ordered.

> The loss is silent unless you count it.

Nothing errors; the numbers are simply smaller.

07

TUNING

MaxAckPending is the loss dial

The cap on messages in flight for one consumer is usually described as a back-pressure knob. On an order-sensitive fold it is something more direct: it sets how many events can race, and therefore how many get refused.

DECISION

What does a low cap do to a large worker pool? **It slows the run and protects the fold.** It does not idle the workers.

MEASURED · 2026-09-16 · NATS 2.14.3 · EIGHT WORKERS, 74 109 EVENTS, -DRAIN

MAXACKPENDING	TIME	RATE	WORKERS THAT ACKED	FOLDED	DROPPED
1	1 m 34.1 s	788/s	8 of 8	74 109	0
3	50.0 s	1 481/s	8 of 8	57 529	16 580 (22.4%)
1000	33.2 s	2 229/s	8 of 8	31 912	42 197 (56.9%)

>

Every worker acked at every cap, including a cap of one with eight workers. No worker was starved of work over the run.

>

A single run is not a constant. The cap-3 row re-run gave 54.6 s and 16 722 dropped. The shape holds; the exact figure is a race.

>

A correction to the common wording. The nats.io worker-pool page says a low cap "starves a large set of workers". At an instant that is visible — num_ack_pending 3 with num_waiting 4-5 means four or five workers are holding a pull request and waiting. Over a whole run it is false: the waiting rotates, and all eight do work.

>

Read it as a dial, not a switch. Cap one is serial and correct. A high cap is fast and loses more than half the log.

HOW TO SET IT

>

Order-free fold: set it high. The cap is then only back pressure.

>

Order-sensitive fold: a cap of one is the only safe value — and it makes the pool pointless, which is the honest signal to partition instead.

>

It is a per-consumer cap, not per worker. Eight workers share one budget.

WHAT IT CANNOT DO

>

It cannot make a pool ordered. It only narrows the window in which racing happens.

>

Low caps cost real time: 1 m 34 s against 33 s here, about 2.8x slower.

>

An instantaneous nats consumer info reading will mislead you. Measure over a run.

08

GOTCHA · DELIVERY

Redelivery after AckWait is not recovery

A worker takes a message and dies without acking. JetStream does the right thing: after AckWait it hands the message to another worker. That is a correct, useful mechanism — and on an order-sensitive fold it arrives too late to be worth anything.

DECISION

Does redelivery repair a fold? **No**. It costs you the whole AckWait and then the fold refuses the event anyway.

MEASURED · 2026-09-16 · NATS 2.14.3 · FOUR WORKERS, ONE MESSAGE ABANDONED WITH -KILL-AT

ACKWAIT	ACTUAL WAIT	HANDED FROM → TO	FOLD MOVED ON BY	REDELIVERED EVENT
30 s	30.02 s	worker 3 → 4	+29 events	Dropped
5 s	5.001 s	worker 4 → 1	+30 events	Dropped

> The wait IS AckWait , to within 20 ms. There is no extra delay and no early detection. The server cannot know the worker is gone; it can only know the clock ran out.

> It bought nothing. While the timer ran, the other workers folded about thirty more events past the abandoned one. The redelivery then arrived behind the state and was refused by BR-OD07 / BR-OD08 — exactly as the rules say it must be.

> A shorter AckWait does not fix this, it only shortens the gap. The redelivered event is still behind by whatever the pool folded in the meantime.

- WHAT REDELIVERY IS FOR
- > An order-free fold, where a late apply is still a correct apply.

> At-least-once work — sending a message, writing a row keyed by event id.

> Surviving a worker restart when nothing else is consuming the same key.

- WHAT IT IS NOT FOR
- > Repairing a projection that has already moved past the event.

> Fast failure detection. Its resolution is AckWait and nothing better.

> Hiding a partitioning problem. The redelivery is a symptom, not a cure.



QUICK REFERENCE

Which card answers your question

Start from the thing you are trying to decide, not from the feature you were reading about.

YOU ARE ASKING...	CARD	SHORT ANSWER
Where does current state live?	01 The log is the only source of truth	In the log. Everything else is derived and can be rebuilt.
Can two read models disagree?	02 Two projections, one log	Yes, and they should. They fold the same log for different questions.
Is a snapshot worth it?	03 What a snapshot buys	Yes past a few thousand events. Replay is linear; a snapshot is flat.
Why is my replay so slow?	04 The ordered consumer that ate the measurement	Check you used <code>Messages()</code> , not <code>Next()</code> .
What happens to a late event?	05 A fold is defined by order	Apply, ignore or refuse. Decide all three in writing.
Can I add workers to go faster?	06 A worker pool buys throughput with correctness	Only if the fold is order-free, or you partition by key.
What should <code>MaxAckPending</code> be?	07 <code>MaxAckPending</code> is the loss dial	Choose it from the fold. It sets how much disorder you accept.
Will redelivery save me?	08 Redelivery after <code>AckWait</code> is not recovery	No. It costs the full wait, then the fold refuses the event.

DEFAULTS WORTH COPYING

- › `LimitsPolicy`, never `InterestPolicy` on a stream you intend to replay. Interest discards a message once every consumer acks it, and the replay then returns nothing, with no error.
- › Streams `SCREAMING_SNAKE`, KV buckets `lowercase-kebab`. A KV bucket name IS a stream name — renaming one orphans the data.
- › Never show a count without its bytes. 100 000 000 events sounds fine until you learn it is 7.6 GB.

TRAPS THIS DEMO WALKED INTO

- › An ordered consumer read one message at a time and made the headline number wrong by two orders of magnitude. Card 04.
- › Ten vehicles hid the race. Round robin over ten keys put two events of one vehicle so far apart that nothing ever collided. Three keys made the lesson visible.
- › A subject that overlapped. `evt.odometer.pool` would sit inside `evt.odometer.>`. A hyphen keeps `ODOMETER_POOL` out of the demo's own filter.



METHOD

Where every number came from

A figure with no date and no machine becomes a constant, and a constant goes stale quietly. Each number in this deck is listed with the day it was taken and the thing it ran on.

THE MACHINE

- › **NATS 2.14.3**, a single server in Docker on a laptop. No cluster, no gateway, no JetStream domain, no operator mode. File storage, `LimitsPolicy` on every stream.
- › Go client, one process, workers as goroutines over **one** durable consumer.
- › Streams: `ODOMETER` (lesson 01), `ODOMETER_POOL` (lesson 02, its own log so the misbehaving consumer never touches the main stream), `ODOMETER_BENCH` (the fixture).

THE NUMBERS

CARD	FIGURE	TAKEN	ON WHAT
01	74 135 events refolded, ten KV documents byte-identical	2026-09-16	<code>ODOMETER</code> , full rebuild of both projections
03	Replay 10 001 events = 25 ms ; snapshot + tail = 1.1 ms	2026-09-16	<code>ODOMETER_BENCH</code> , one vehicle, best of three
03	2 900 events of tail in 12 ms	2026-09-16	the same command, run mid-catch-up
04	8.2 s before the fix, 25 ms after	2026-09-16	the same replay, <code>Next()</code> then <code>Messages()</code>
06	1 / 2 / 4 / 8 workers: 23.5 / 12.2 / 6.3 / 4.8 s	2026-09-15	<code>ODOMETER_POOL</code> , 10 029 events, <code>-drain</code>
07	caps 1 / 3 / 1000: 1 m 34.1 s / 50.0 s / 33.2 s	2026-09-16	<code>ODOMETER_POOL</code> , eight workers, 74 109 events
07	cap 3 re-run: 54.6 s, 16 722 dropped	2026-09-16	the same run repeated, to show it is a race
08	waits of 30.02 s and 5.001 s	2026-09-16	four workers, one message abandoned by <code>-kill-at</code>

HOW TO READ THEM

- › **Read the shape, not the value.** A laptop is not your production cluster. "Replay is linear, a snapshot is flat" transfers; "25 ms" does not.
- › **A single run is a sample.** Where a figure is a race, the deck shows a second run.
- › **One number here has already been retracted.** Card 04 is that retraction, kept on purpose.
- › **The screens do not print these.** Every number in the demo's UI comes from a run the reader just made. This page is the only place a constant is allowed to live.

Sources in the repo: [demos/04-jetstream-cQRS/README.md](#) · [docs/LESSON-02.md](#) · [BUSINESS_RULES-ODOMETER.md](#) · [docs/Demo-04-Plan.md](#) · [cQRS/domain.go](#)